

Prosjekt Del 1: ER-modell og relasjonsdatabaseskjema

a) Forutsetninger og restriksjoner:

1. Det antas at ingen bemanningsintervaller for samme senter har samme starttidspunkt på samme ukedag. Dermed er kombinasjonen (senter, ukedag, start_tidspunkt) tilstrekkelig for å identifisere en unik bemanningsrelasjon. En bemanning kan ikke eksistere uten et senter, og er dermed en svak entitet.
2. Det antas at hvert senter har én unik åpningstid per ukedag. En åpningstid kan heller ikke eksistere uten et senter, og er dermed en svak entitet.
3. Det antas at e-post, navn og mobilnummer ikke er unike identifikatorer for bruker. Derfor benyttes en kunstig primærnøkkel (id) for å identifisere en bruker entydig.
4. Det antas at fasilitetstyper (f.eks. spinning, yoga, badstue) har unike navn, og at navnet derfor kan benyttes som identifikator for fasilitet.
5. Det antas at aktivitetsnavn (f.eks. spin4x4, spin45) er unike, og at navnet dermed kan brukes som identifikator for aktivitetstype.
6. Det antas at det kan holdes flere gruppetimer i samme sal over tid. Derfor benyttes en egen id for å identifisere hver gruppetime. Tidspunktet for når påmelding åpner kan beregnes som starttid – 48 timer, og lagres ikke eksplisitt.
7. Det antas at en bruker maksimalt kan melde seg på én gang per gruppetime. Dersom brukeren ikke møter, settes oppmøte_tidspunkt til NULL.
8. Det antas at en bruker maksimalt kan få én prikk per gruppetime, og at en prikk ikke kan eksistere uten en tilhørende påmelding.
9. Det antas at en bruker ikke kan ha overlappende svartelisteperioder, og at det ikke kan eksistere to svartelister med samme startdato for samme bruker. Sluttdato lagres ikke eksplisitt, men beregnes som startdato + 30 dager.
10. Det antas at sykkelnummer er unikt innen en sal, og at hver sykkel tilhører nøyaktig én sal. En sykkel kan heller ikke eksistere uten en sal, og er dermed en svak entitet.
11. Det antas at møllenummer er unikt innen en sal, og at hver tredemølle tilhører nøyaktig én sal. En tredemølle kan heller ikke eksistere uten en sal, og er dermed en svak entitet.
12. Det antas at en reservasjon gjelder én bestemt gruppe og én bestemt sal. Ukedag refererer til en fast, ukentlig gjentakende reservasjon.

b) Tabeller og normalform:

Se SQL-script for tabeller

For å vise at relasjonene er på BCNF-form, kan vi starte med de enkleste relasjonene. Vi starter med å definere BCNF som alle relasjoner som er slik at for enhver ikke-triviell funksjonell avhengighet $X \rightarrow Y$, så er X en supernøkkel.

I flere av tabellene finnes det kun én ikke-triviell funksjonell avhengighet, nemlig primærnøkkelen som bestemmer alle øvrige attributter. Siden determinanten er en supernøkkel, oppfyller disse relasjonene BCNF.

Vi undersøker om de resterende tabellene er på BCNF:

1. *Aktivitetstype*-tabellen har navnet på aktivitetstypen som primary key og navnet på fasiliteten som foreign key. Da har vi funksjonell avhengighet navn $\rightarrow$ beskrivelse, fasilitet_navn. Navn er primary key, og derfor også superkey, og BCNF er oppfylt.
2. *Gruppetime*-tabellen har id som peker på aktivitetstype, instruktør og sal. Funksjonelle avhengigheter $X \rightarrow Y$ er da kun hvor gruppetime sin id er x, og denne er superkey så BCNF er oppfylt.
3. *Reservasjon* har primærnøkkelen id. Den eneste ikke-trivielle funksjonelle avhengigheten er id $\rightarrow$ sal_id, gruppe_id, starttid, varighet, ukedag. Siden determinanten er en supernøkkel, oppfyller relasjonen BCNF.
4. *Sal*-tabellen peker på sit_senter, og den eneste funksjonelle avhengigheten er sal sin id som peker på alt annet. Ingen attributter hos sit_senter bestemmer id til sal, og sal_id er en superkey som følge av at det er en primary key. Derfor er også denne relasjonen på BCNF.
5. I *bemannings*-tabellen har vi en funksjonell avhengighet (sit_senter_navn, ukedag, start_tidspunkt) $\rightarrow$ slutt_tidspunkt, men (sit_senter_navn, ukedag, start_tidspunkt) er primary key og derfor også superkey, og BCNF er oppfylt.
6. For tabellen *åpningstid* er den eneste funksjonelle avhengigheten (sit_senter_navn, ukedag) $\rightarrow$ åpner, stenger. Altså at riktig sit_senter og ukedag peker på når senteret åpner og stenger. (sit_senter_navn, ukedag) er primery key i tabellen, derfor superkey og BCNF er oppfylt.
7. For *sykkel* og *tredemølle* har begge tabellene hhv primary key (sykkel_nr, sal_id) og (mølle_nr, sal_id) som primary key. Denne peker på alle resterende attributter i *sykkel* og *tredemølle*. BCNF er derfor oppfylt for disse relasjonene.

8. *Svartelista*-tabellen har primary key (*bruker_id*, *start_dato*), og svartelista har ingen andre funksjonelle avhengigheter enn de som går fra denne primærnøkkelen til en del av seg selv, så BCNF er trivielt for denne relasjonen.
9. *Medlemskap*-tabellen har primary key (*bruker_id*, *gruppe_id*) fra *bruker* og *gruppe*, og har kun trivielle funksjonelle avhengigheter. BCNF er derfor oppfylt.
10. *Påmelding*-tabellen har funksjonell avhengighet (*bruker_id*, *gruppetime_id*) $\rightarrow$ *oppmøte_tidspunkt*, (*bruker_id*, *gruppetime_id*) er primary key, og også superkey og BCNF er oppfylt.
11. *Senter_fasilitet*-tabellen har primary key (*fasilitet_navn*, *sit_senter_navn*) som er alle-til-alle relasjonen mellom *sit_senter* og *fasilitet*. Denne har kun trivielle funksjonelle avhengigheter (peker bare på deler av primary key), og er derfor på BCNF.
12. *Senterbesøk*-tabellen har primary key (*bruker_id*, *sit_senter_navn*, *ankomst*), og vil også kun ha trivielle funksjonelle avhengigheter og være på BCNF.
13. *Prikk*-tabellen har primærnøkkel (*bruker_id*, *gruppetime_id*), som også er en fremmednøkkel til påmelding. Den eneste ikke-trivielle funksjonelle avhengigheten er (*bruker_id*, *gruppetime_id*) $\rightarrow$ *dato_registrert*. Siden determinanten er relasjonens primærnøkkel, og dermed en supernøkkel, oppfyller relasjonen BCNF.

c) **Script og restriksjoner:**

Se SQL-script for databasen, primærnøkler og fremmednøkler.

Følgende regler kan ikke håndheves direkte gjennom ER-modellen, men må implementeres i applikasjonslogikk eller databasebegrensninger:

- En instruktør kan ikke undervise to gruppetimer som overlapper i tid.
- En deltaker kan ikke være påmeldt to gruppetimer som overlapper i tid.
- Avbestilling må skje senest én time før start.
- Oppmøte må registreres senest fem minutter før start.
- Maksimalt tre prikker innen en periode på 30 dager fører til svartelisting.

d) **Spørsmål:**

1. For å sikre at en instruktør ikke kan være til stede på to forskjellige plasser samtidig, må vi sørge for at de gruppetimene som er tilordnet en instruktør ikke overlapper i tid. Dette kan ikke uttrykkes direkte i ER-modellen, ettersom modellen kun kan representere

strukturelle begrensninger som nøkler og kardinalitet, og ikke tidsmessige overlapp mellom intervaller.

Begrensningen må derfor implementeres på databasesiden ved hjelp av for eksempel triggerer eller mer avanserte check-constraints som kontrollerer overlappende tidsintervaller ved innsetting eller oppdatering av gruppetimer. Alternativt kan den håndheves i applikasjonslogikken (for eksempel i Python) før en gruppetime opprettes.

2. Det samme gjelder for kravet om at en bruker ikke kan være til stede på to forskjellige plasser samtidig. Her må man forhindre at en bruker melder seg på to gruppetimer med overlappende tidsintervaller.

Dette kan heller ikke uttrykkes direkte i ER-modellen, da den kun støtter strukturelle begrensninger som nøkler og kardinalitet, og ikke tidsbaserte restriksjoner mellom ulike forekomster. Begrensningen må derfor implementeres i programvaren, enten ved hjelp av triggerer eller check-constraints i databasen som kontrollerer overlapp ved påmelding, eller i applikasjonslogikken før en påmelding registreres.

3. Det testes for utestengelse når en bruker forsøker å booke en trening (brukstilfelle 2). Programmet må da kontrollere om det finnes en aktiv svartelisteperiode for den aktuelle brukeren før bookingen gjennomføres. Dette kan gjøres ved å undersøke om det eksisterer en rad i svarteliste-tabellen der start_dato er innenfor de siste 30 dagene (det vil si at utestengelsesperioden fortsatt er aktiv).

Opprettelse av utestengelse skjer etter følgende logikk: Dersom en bruker ikke møter opp til en gruppetime og det opprettes en prikk, må systemet kontrollere hvor mange prikker brukeren har mottatt i løpet av de siste 30 dagene. Hvis antallet er tre eller flere, opprettes en ny rad i svarteliste-tabellen med dagens dato som startdato.

Denne logikken kan implementeres som en del av brukstilfelle 6, eller som en automatisk kontroll i databasen (for eksempel ved hjelp av triggerer) eller i applikasjonslogikken når en prikk registreres.

4. Ved første vurdering fremstår det som unødvendig og redundant å lagre statistikk eksplisitt, ettersom statistiske størrelser allerede kan utledes fra dataene som er lagret i databasen. Aggregert informasjon som antall deltakelser, oppmøtefrekvens og antall

gjennomførte gruppetimer kan beregnes direkte ved hjelp av SQL-spørringer basert på eksisterende relasjoner.

Fordelen ved å lagre statistikk eksplisitt oppstår primært i systemer med svært store datamengder eller høye krav til ytelse. I slike tilfeller kan komplekse spørringer med JOIN, GROUP BY og aggregeringsfunksjoner være ressurskrevende. Forhåndsberegnet og lagret statistikk kan da redusere responstid og avlaste databasen. Et annet moment er bevaring av historiske nøkkeltall dersom underliggende rådata slettes eller arkiveres.

Ulempene ved eksplisitt lagring av statistikk er imidlertid betydelige. Man introduserer redundans, noe som øker risikoen for inkonsistens dersom statistikken ikke oppdateres korrekt ved hver endring i underliggende data. Dette krever tilleggsmekanismer som trigger eller applikasjonslogikk for å sikre konsistens. En slik denormalisering kan også innebære brudd på BCNF, ettersom man lagrer avledet informasjon som funksjonelt avhenger av andre attributter.

Ved å beregne statistikk dynamisk gjennom spørringer unngår man redundans og bevarer en fullt normalisert modell. Dette sikrer dataintegritet og gjør systemet mer robust mot inkonsistens. Ulempen er at ytelsen kan reduseres ved svært store datamengder, men dette er ikke en kritisk faktor i vårt brukstilfelle.

I dette prosjektet er det derfor verken nødvendig eller hensiktsmessig å lagre statistikk eksplisitt. Systemet skal håndtere brukstilfeller som besøkshistorikk (brukstilfelle 5) og oversikt over brukere med flest gjennomførte gruppetimer (brukstilfelle 7). All nødvendig rådata finnes allerede i tabellene for Bruker, Gruppetime og Påmelding. Det er dermed både mest presist, konsistent og i tråd med normaliseringsprinsipper å generere statistikken ved behov ved hjelp av SQL-spørringer.